Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	Envío de mensajes distribuido
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	Uno
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Nombre del Estudiante	
Fecha	29 de septiembre de 2026 02 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo

3. Materiales y reactivos:

Diapositivas, y enunciados del problema:

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y herramientas

Computadora personal con conexión a internet

Guía de práctica.

Plantilla de matriz comparativa

5. Procedimiento / Metodología

Fase 0 (10 min). Formación de equipos de 4 (asignación del docente).

Fase 1 – Investigación (50 min). Cada integrante investiga su arquitectura y completa su sección de la matriz (estructura, ventajas, desventajas, escalabilidad, ejemplos). El equipo elabora el diagrama de su arquitectura en draw.io.

Fase 2 – Socialización (30 min). Cada equipo expone 5 min. El resto completa la matriz comparativa con la información expuesta (una sola matriz, grupal).

Fase 3 – Caso de estudio (55 min). Cada equipo selecciona la arquitectura más adecuada para el enunciado, la justifica con mínimo 3 argumentos técnicos, elabora el diagrama de la solución e identifica dónde se aplican transparencia, escalabilidad y tolerancia a fallos.

Fase 4 – Reporte y entrega (25 min). Cada estudiante redacta sus conclusiones individuales; el equipo compila la matriz final y los diagramas. Entrega en la plataforma indicada.

Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
Estructura	Aplicación dividida en dos componentes principales: clientes y servidores, el cliente realiza la solicitud de servicios o datos, mientras que el servidor proporciona los recursos. Comunicación constante mediante protocolos de red, permitiendo un flujo bidireccional entre ambos.	La red P2P se organiza en cuatro modelos principales: P2P estructurada que usa DHT (Tablas de Hash Distribuidas) para asignar datos a nodos en específico. P2P No Estructurada, los nodos están conectados aleatoriamente o ad-hoc. P2P Híbrida (Super-nodos/Peer-assisted), un nodo central coordina la información y la transferencia de	La arquitectura multicapa organiza una aplicación en varias capas horizontales, donde cada una de ellas tiene un rol específico, siguiendo el principio de separación de responsabilidades.	La estructura utilizada en microservicios se basa en el desglose de componentes independientes que se pueden desplegar en distintos servicios y se comunican a través de una API. Por esta razón se puede desglosar en 3 partes fundamentales, el servicio, el gateway u orquestados y el cliente o consumidor.

		datos se realiza entre los nodos. P2P Descentralizada/Pura, no existe nodo central sino nodos similares conectados entre sí.		
Ventajas	- Centralización de datos en el servidor, lo cual garantiza que todos los clientes accedan a la misma información. - Reducción de conflictos de información. - Integridad y consistencia de datos.	- No depende de un solo servidor. - Tolerancia ante fallo, ya que ante el fallo de un dispositivos no afecta a los demás. - Los dispositivos (nodos) pueden compartir archivos e información entre ellos, esto ayuda a que no solo un dispositivo tenga más carga de trabajo. - Optimización y escalabilidad, cuando hay muchos usuarios la red se expande en cuanto a su almacenamiento, procesamiento y ancho de banda libre sin el uso de servidores más potentes.	-La separación de responsabilidades permite que cada módulo se enfoque en la lógica de su propio nivel. - El aislamiento permite que, si se produce algún error por alguna modificación , este no se propague a otras capas. -Se pueden hacer testability ya que al estar aisladas se pueden hacer pruebas o simulaciones con capas adyacentes mediante objetos falsos(mocks o stubs)	- Libertad de elección de tecnologías para cada servicio. - Facilidad de mantenimiento especializado en cada servicio o componente. - Gran Resiliencia permitiendo un funcionamiento adecuado si algún componente falla.
Desventajas	- Dependencia del servidor, esto implica que los clientes se vean afectados si el servidor presenta algún fallo o caída que interrumpa el flujo de trabajo de una organización, empresa, generando pérdidas, tanto económicas como de tiempo. - La gestión de actualizaciones en esta arquitectura representa un desafío mayor, ya que implica mantener a los clientes sincronizados con el servidor.	- Seguridad y control de datos, al no tener un nodo central se dificulta la autenticación, verificación o prevenir malware. - Rendimiento inestable, todos los nodos no tienen las mismas condiciones como el ancho de banda (conexiones rápidas) o recursos. - Sincronización de datos al mismo tiempo en cada uno de los nodos.	-El antipatrón Architecture Sinkhole ocurre cuando muchas peticiones atraviesan todas las capas de la aplicación, actuando algunas de ellas únicamente como procesos de paso. -Riesgo de bloqueo tecnológico, ya que cada capa es una unidad de gran tamaño y cambiar un	- Alta complejidad de gestión de múltiples servicios. - Coherencia o consistencia de datos. - Costes de mantenimiento o despliegue de cada servicio.

	- Problemas de concurrencia, a medida que aumente el número de clientes el servidor debe ser capaz de manejar estas solicitudes efectivamente.		lenguaje de programación, framework o base de datos requiere más esfuerzo comparado con los estilos del microservicio.	
Escalabilidad	- Desarrollo independiente, se puede trabajar en la implementación de mejoras en uno sin afectar al otro (cliente o servidor), favoreciendo la evolución de aplicaciones. - Incremento de capacidad del servidor, permitiendo a las empresas adaptarse a la demanda de usuarios de forma eficiente.	- Cuando la red crece en número de nodos, el sistema escala de forma horizontal, cada nodo tiene sus propios recursos sin la necesidad de tomar algún servidor. - El incremento de nodos ayuda a la capacidad total del procesamiento, almacenamiento y ancho de banda.Esto permite escalar a un nivel masivo de peticiones evitando saturaciones.	-Se puede decir que la escalabilidad de esta arquitectura es de nivel intermedio, ya que supera lo que es un monolito, pero no llega a tanto como el de microservicios, pero como tal tiene su fortaleza en la descomposición de sus capas, la cual permite tener un mejor control de la organización, mantenimiento y funcionamiento del sistema.	- Esta arquitectura aporta una alta escalabilidad gracias a su sistema de división de tareas o servicios independientes
Ejemplos reales	- Netflix, YouTube son dependientes de una red cliente servidor, mediante la cual procesan solicitudes y gestionan el contenido y distribución del contenido multimedia. - Tiendas digitales (ecommerce), al ingresar a la dirección web de una tienda o plataforma de streaming, el navegador actúa como cliente, el cual enviara la solicitud al servidor web de la tienda, el servidor procesara la solicitud, encontrara y retornara la información solicitada en forma de página web.	- Bitcoin y Criptomonedas están conformados por miles de nodos a nivel mundial, no hay un un servidor central cada nodo tiene una copia idéntica de todo el historial de transacciones. - BitTorrent es un protocolo de transferencias de archivos pesados, cuando se descarga algún archivo no se descarga de un nodo central sino de las computadoras de otras personas (peers) que tiene este archivo, por lo que se descarga un fragmento de cada uno y al mismo tiempo se suben estos fragmentos para otros.	-Herramientas de software y bibliotecas como ConStore y librería Java; estas son aplicaciones y librerías donde sus módulos se organizan en capas jerárquicas. -Health Watcher Application, este es un sistema web de gestión de salud donde los usuarios registran quejas y consultas de salud y las organizaciones procesan las respuestas; está dividido en capa de presentación, capa de negocio, capa de datos y capa de modelo.	- Servicios como Spotify utilizan una arquitectura de microservicios principalmente para mejorar su calidad de servicio y la eficiencia de respuesta en todo el mundo. - Un servicio financiero como J.P Morgan logró una arquitectura de microservicios en la nube para la flexibilidad que necesita un sistema financiero y no depender de un único proveedor, logrando así mantener un servicio confiable a toda hora.

Fase 5: Socialización y matriz comparativa

1. Cada equipo expone su arquitectura durante 5 minutos (proyector)
2. Durante las exposiciones, todos los equipos llenan la matriz comparativa con la información de las cuatro arquitecturas:

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
	Estructura	Se menciona que esta compuesta por cliente que hace solicitudes y servidor que las atiende.	Se mencionó que es una conexión de red entre nodos que pueden actuar como cliente o servidor	Se mencionó que este modelo trabaja en capas, mediante la separación de responsabilidades.	Se menciono acerca de la operabilidad independiente de los servicios y con una comunicación por mensajes. La importancia de las funciones

GRUPO 1	Ventajas	Se menciono la centralización y facilidad de gestión.	Alta escalabilidad, privacidad y propiedad compartida de cada nodo.	Se mencionó que al aplicar multicapa, pues se tendrían tiempos de respuesta más óptimos.	- No se necesita detener la aplicación para aplicar cambios. - facilidad de mantenimiento
	Desventajas	Se mencionó la dependencia del servidor y posibles fallos.	Más volatilidad de nodos	Si se mueve o cambia algo en alguna capa, será un poco más costoso de llevar a cabo.	- Mayor complicación del funcionamiento. - Aumenta la demora del servicio. - Demora de mantenimiento.
	Escalabilidad	Se mencionó que permite aumentar recursos según la demanda.	Ilimitada y global	Al estar aislado por responsabilidades, en tal caso se tendría mejor control en cada capa para realizar algún cambio.	- Aumento de servicios por necesidad.
	Ejemplos reales	Sistema educativo de matriculación.	BitTorrent	Sistema de un banco.	Se mencionó a Netflix y CloudSound.
GRUPO 3	Estructura	El servidor espera peticiones del cliente y las procesa.	Puede actuar como cliente o servidor al mismo tiempo o hibrido	Se menciona que cada capa trabaja individualmente.	Se indico que se trata de la union de pequeños servicios
	Ventajas	Información centralizada.	No tiene un nodo principal, los nodos pueden actuar como cliente o servidor	Se menciona que es más factible aplicarlo en proyectos grandes.	cada microservicio puede ser actualizado o editado sin afectar todo el sistema - Mayor complejidad de manejar datos
	Desventajas	En caso de fallar el servidor afectaría al cliente.	Alta complejidad y riesgo a saturación y tráfico de red.	Como contraparte, si es en proyectos pequeños, se puede optar por otro modelo.	- posible inconsistencia de datos - Costos mayores de mantenimiento o utilización de cada servicio
	Escalabilidad	Capacidad de mejorar el servidor.	Entre más nodos hay en una red es más escalable y robusta.	Se menciona que si se trabaja de esta forma se mantiene un control organizado del proyecto.	- Escalabilidad horizontal y escalabilidad vertical
	Ejemplos reales	Servicios online como bancos y páginas web.	Criptomonedas	Se menciona el ejemplo de un banco, cuya capa más importante es la de datos.	- Uber y Amazon.
Grupo 4	Estructura	El navegador web es el cliente y el servidor responde.	Existen varios modelos como es híbrido,	Se menciona que varias empresas utilizan este modelo	Basada en 3 partes, los servicios, el orquestador y la interfaz. aplica el pensamiento

			descentralizado, estructurado y no estructurado	aplicando cpap de presentación, interfaz media y la cpa de almacenamiento.	de “Divide y vencerás”.
	Ventajas	Desacoplamiento, únicamente el servidor procesa las solicitudes.	Tolerante a fallos en caso que un nodo falle los otros trabajan normalmente	Se menciona que estas hacen tareas específicas y funciones independientes sin afectarse entre capas.	-Diversidad tecnológica.
	Desventajas	Servidor puede colpasar si no se tiene los recursos necesarios.	Volalitabilidad nodos o Churm debido a la entrada y salida de usuarios.	Se menciona de igual forma que al hacer cambios de alguna índole se solicitarán más recursos y trabajo.	- Necesidad e implementar mayor seguridad aumentando costos de operatividad.
	Escalabilidad	Se puede agregar mas recursos al sistema.	Entre más nodos o dispositivos estén en una red, mayor es el procesamiento y el almacenamiento.	Es escalable por su arquitectura de capas.	- Limitada por el presupuesto
	Ejemplos reales	Sistema de universidad.	Gnutella, funciona de forma descentralizada, buscando archivos entre los usuarios.	Sistema de matrículas.	- Billeteras digitales
Grupo 5.	Estructura		Red donde se interconectan los nodos entre sí, actuando como cliente o servidor.	Se menciona que su estructura está dividida en capas de forma horizontal en las cuales se definen responsabilidades.	
	Ventajas		Aunque varios nodos se desconecten de miles que hayan no hay fallos, los que tienen conexión siguen funcionado	Se puede trabajar de forma independiente sin afectar a otras capas.	
	Desventajas		Al realizar una búsqueda	Si se necesita hacer algún	

			de información, muchos nodos pueden estar buscando entre sus nodos cercanos repetidamente que puede ocasionar tráfico o saturación de la red.	cambio, pues se gastarán más recursos.	
	Escalabilidad		La estabilidad de la red depende de la cantidad de nodos funcionales.	Se menciona que es escalable por la arquitectura de capas, ya que trabajan de forma independiente.	
	Ejemplos reales		Windows Delivery Optimization, cuando sube una actualización nuestra computadora la puede descargar desde otras computadoras así mismo como de nuestra computadora pueden descargar otras	Aplicación web de banca en línea.	- Uber
Grupo 6	Estructura		Cada nodo puede compartir entre sí recursos archivos o almacenamiento.	Se menciona que se maneja en capas que trabajan de forma aislada sin afectar a otras capas.	
	Ventajas		Comparten recursos directamente.	Se puede realizar un mantenimiento aislado.	
	Desventajas		Para la administración de este tipo de red puede ser más compleja o difícil.	Se menciona que su complejidad aumenta y puede tener un alto riesgo de dependencia acorde con el proyecto.	

	Escalabilidad		Cuando hay conexiones variadas,es decir entran y salen puede afectar el rendimiento de la red.	Es escalable por su arquitectura en capas, ya que su trabajo no afecta a otras adyacentes.	
	Ejemplos reales	Servicios onlines como tiendas online, páginas web.	Cuando se descarga una película se realiza por fragmentos, si uno de los nodos falla o deja de estar disponible, los fragmentos faltantes pueden obtenerse de otros nodos de la red, permitiendo continuar la descarga sin depender de un único equipo.	Delivery Se menciona que tiene capa de presentación que son los pedidos, lógica de negocio que es la cuenta de pedidos, y la base de datos que es el stock de productos.	- envío de pedidos
Grupo 7	Estructura		Esta formada por múltiples nodos o tambien llamados peers interconectados.	Se menciona que esta se maneja con una capa de presentación, que son los usuarios, una capa de aplicación, que es el frontend, y una capa de datos, que son las bases de datos.	
	Ventajas	Sincronización de datos.	Reduce en parte costos de infraestructura.	Se menciona que las capas trabajan de forma aislada para no afectar a otras capas.	
	Desventajas	Cuellos de botella.	Los nodos pueden desconectarse	Si se hace algún cambio de índole importante, puede requerir más esfuerzo y costo para lograr ese cambio.	
	Escalabilidad		Se puede agregar más nodos en la red sin afectar a los	Al trabajar en capas se mantienen aislados los diferentes	

			demás	módulos y así se puede trabajar de una forma más controlada.	
	Ejemplos reales	Aplicación web, servicio de gmail.	Ethereum, realiza transacciones y hace trabajos inteligentes sin depender de un servidor central.	Tienda universitaria.	- Amazon

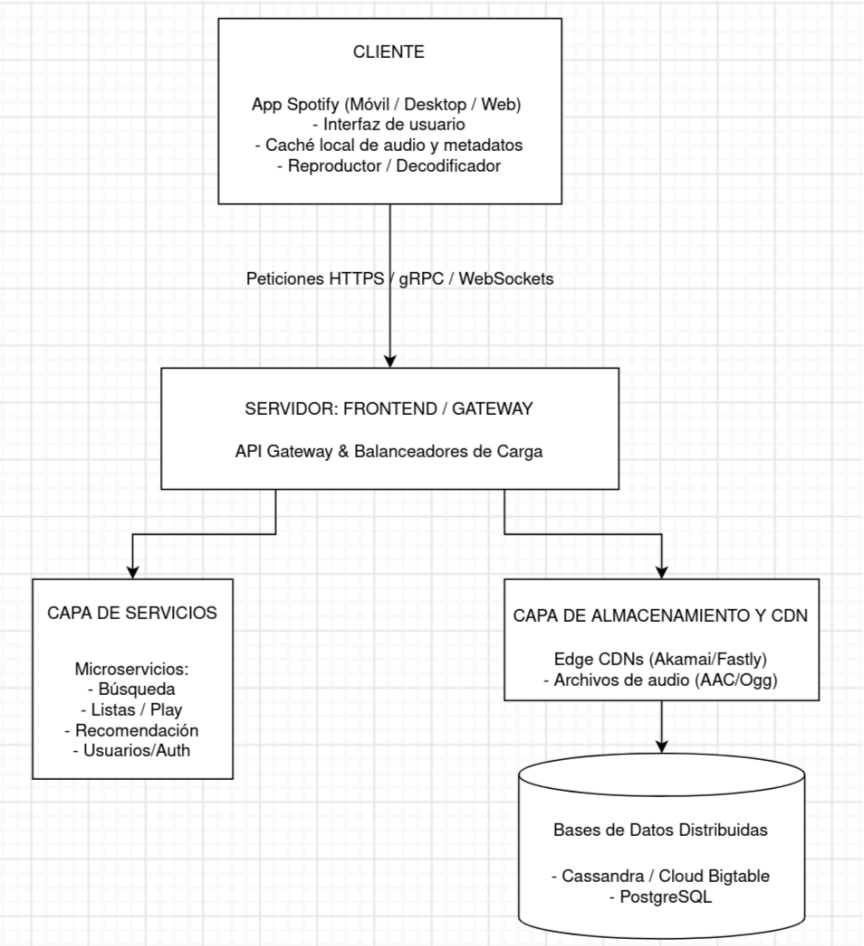
Grupo 8	Estructura				
	Ventajas				
	Desventajas	Saturación del servidor antes una cantidad masiva de solicitudes.			
	Escalabilidad				
	Ejemplos reales	Sistema bancario			

Caso de estudio en arquitectura "Cliente-Servidor"
Caso de estudio: **Spotify (Plataforma de Streaming de Música)**

Spotify opera bajo un modelo **Cliente-Servidor descentralizado por microservicios**.

- **Cliente:** La aplicación móvil (iOS/Android), de escritorio (Windows/macOS), reproductor web o clientes integrados (Smart TVs, autos). Se encarga de la interfaz de usuario, la decodificación de audio en tiempo real, el almacenamiento en caché local y el envío de peticiones de usuario.
- **Servidor:** La infraestructura del backend alojada en la nube (Google Cloud Platform), compuesta por APIs (API Gateway), microservicios especializados (autenticación, catálogo, recomendaciones, pagos) y una red de distribución de contenido (CDN) para mantener los archivos de audio cifrados.

Diagrama:



Justificación técnica:

Spotify cumple rigurosamente con los principios de la arquitectura Cliente-Servidor por los siguientes aspectos:

- **Asimetría en las funciones:** Existe una distinción clara de roles. El cliente es reactivo (inicia la comunicación según la interacción del usuario), mientras que el servidor es pasivo/reactivo (espera peticiones, valida credenciales y entrega respuestas).
- **Control centralizado del estado y derechos (DRM):** La lógica de negocio sensible, el procesamiento de licencias de reproducción, las métricas de regalías y el catálogo oficial residen exclusivamente en los servidores. El cliente nunca posee la biblioteca de forma independiente ni los archivos de audio sin cifrar.
- **Protocolos estándar de solicitud-respuesta y streaming:** La comunicación utiliza HTTPS, WebSockets para notificaciones/control en tiempo real (por ejemplo, Spotify Connect) y llamadas gRPC para comunicación entre el cliente y el backend de alto rendimiento.

Transparencia:
En este caso la transparencia hace referencia a la capacidad de ocultar la complejidad de la infraestructura al cliente.
Por ejemplo la ubicación de los servidores, la forma de acceso a los recursos y el balanceador de carga para atender el tráfico de los clientes.

Escalabilidad:

Spotify mantiene una escalabilidad horizontal y una distribución desacoplada, usando CDNs para la distribución de carga pesada, servidores sin estado y el particionamiento de sus bases de datos para escalar la lectura y escritura de manera individual.

Tolerancia a fallos:
Como ya se ha mencionado, Spotify trabaja de una forma casi totalmente desacoplada, uniendo funcionalidades como la redundancia multi-region, patrones de Circuit Breaker y hasta datos caché a nivel del cliente, esto debido a que tienen una premisa de que cualquier componente puede fallar y ser reemplazado temporalmente.

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).
- Un diagrama digital de la arquitectura asignada (draw.io o similar).
- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).
- Diseño del caso de estudio: diagrama + justificación técnica (mínimo 3 argumentos) + identificación de transparencia, escalabilidad y tolerancia a fallos.
- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

7. Preguntas de Control:

1- ¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?

Un sistema distribuido consta de la división de tareas o la modularidad en las actividades que realiza el sistema, esta es la principal diferencia contra un sistema centralizado en la que todo el funcionamiento se encuentra en un mismo punto centralizado el cual si falla el servicio se ve afectado todo el sistema, cosa que no pasa en un sistema distribuido que tiene gran resiliencia a fallos.

2- Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.

La arquitectura cliente-servidor se basa en que los clientes que quieran consumir el servicio van a utilizar un mismo servidor, es por ello que este mismo puede sufrir por la gran concurrencia que pueda existir por lo que si falla este único servidor todo el sistema o el servicio cae o falla.

Por otro lado la arquitectura peer-to-peer mantiene una comunicación de “todos con todos” entre los nodos conectados, esto mejora mucho la accesibilidad a los datos necesarios de un servicio ya que todos constan con los datos necesarios y se puede obtener de cualquier lado el dato necesario, por lo que si cae un nodo o no está disponible se puede realizar la tarea o obtención del dato de cualquiera de los nodos.

3- ¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?

El middleware es como un traductor y coordinador entre aplicaciones. Cada una de las aplicaciones debe tener un centro de middleware para que este pueda gestionar soplicitudes, aplicar reglas y enrutar información de manera segura y eficiente.El middleware está en medio del sistema operativo entre el hardware y la lógica de negocio, que son aplicaciones. Entonces, este actúa como intermediario para que no aprendan el idioma, por ejemplo, para poder comunicarse con cada base de datos o cada servicio, solo pasan por el middleware y este decide dónde va cada mensaje, traduce formatos y avisa si algo falla.

4- En la arquitectura de tres capas (3-tier), ¿qué función cumple cada capa y por qué separarlas mejora el mantenimiento?

La primera capa es la de presentación. Esta maneja solo la interacción con el cliente y también se le presenta de forma visual la información.

La segunda capa es la capa de aplicación. Está a cargo de la coordinación de las tareas del sistema, dirige cómo hacer el flujo de trabajo y también ejecuta la lógica que se necesita para cumplir los casos de uso.

La tercera capa es de datos. Esta es la responsable de almacenar, gestionar y también recuperar la información en el sistema.

Así que, al separar las capas, se mantiene el principio de responsabilidades, ya que cada capa cumple una función lógica específica; así, si existe algún cambio en una capa, solo se trabajará en esa y no afectará a otra adyacente.

5- Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?

Los microservicios se dividen en partes pequeñas llamados servicios a una aplicación, los mismos son independientes. La principales ventajas es la escalabilidad ya que se al dividirse cada parte administra un módulo en específico sin duplicarse, además en caso de un fallar algún servicio este se congela evitando afectar a los demás. Otra ventaja es el dar mantenimiento ya que más fácil dar actualizaciones sin necesidad de desplegar toda la aplicación a diferencia de monolítico que es una sola parte.Sin embargo tiene desafíos como al ser muchos nodos tiene mayor complejidad para su mantenimiento y sincronización de datos al mismo que todos tengan un mismo nivel de recursos.

6- ¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?

La arquitectura Peer-to-Peer ya que BitTorrent permite que los usuarios compartan de manera directa fragmentos de archivos entre sus dispositivos, sin depender de un único servidor. Ayudando a distribuir la carga y optimizando recursos de otros dispositivos y continuando con la descarga en caso de que algún nodo esté desconectado.

7. ¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?

La escalabilidad vertical influye de manera positiva en arquitecturas como cliente-servidor, monolíticas, ya que implica ampliar los recursos de un solo nodo, no obstante esto puede llegar a presentar límites económicos.

La escalabilidad horizontal favorece arquitecturas como microservicios peer to peer, ya que permite repartir la carga entre múltiples máquinas

(agregación de nodos), lo cual significa una mayor tolerancia a fallos, disponibilidad y crecimiento ante la demanda.

8. ¿Qué significa transparencia en un sistema distribuido y mencione dos tipos de transparencia?

Esta es una propiedad la cual oculta la naturaleza distribuida del sistema al usuario y desarrollador, esto significa que se presenta como una entidad única, existen dos tipos de transparencia, **la transparencia de ubicación**, está ocultara la localización física de los recursos, y la **transparencia de fallos,** la cual oculta los errores y recuperaciones haciendo posible que el servicio continúe operativo.

Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?

En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?

8. Conclusiones

- Cada una de las arquitecturas estudiadas tienen su ventajas como desventajas, en cuanto a P2P se comprende que es una red extensa con nodos interconectados entre sí, donde a mayor nodos es más robusta es la red, principalmente se diferencia su estructura por ser estructurada, no estructurada, híbrida o descentralizada. Además de en cada de lagun nod lo demás funciona con normalidad

- Al aprender sobre Multicapa, se entendió por qué se aplica en la mayoría de las empresas, ya que su lógica es trabajar en capas de forma aislada y no representa un problema a la hora de hacer algún tipo de cambio.

9. Evaluación

No aplica en esta practica

10. Bibliografía

- Diapositivas

[1] «Patrones De Arquitectura De Microservicios, ¿Qué Son Y Qué Ventajas…», www.paradigmadigital.com. Accedido: 29 de septiembre de 2026. [En línea]. Disponible en: https://www.paradigmadigital.com/dev/patrones-arquitectura-microservicios-que-son-ventajas/

[2] «What Are Microservices?», microservices.io. Accedido: 29 de septiembre de 2026. [En línea]. Disponible en: https://microservices.io

[3] Atlassian, «Arquitectura De Microservicios», Atlassian. Accedido: 29 de septiembre de 2026. [En línea]. Disponible en: https://www.atlassian.com/es/microservices/microservices-architecture

[4] «Service Not Allowed», Google.com. Accedido: 29 de septiembre de 2026. [En línea]. Disponible en: https://cloud.google.com/learn/what-is-microservices-architecture?hl=es

[5] O. Söderman, «Comparison of web application architecture styles», *Aaltodoc*, Aalto University. [En línea]. Disponible en: https://aaltodoc.aalto.fi

[6] «Redefining measures of Layered Architecture», *arXiv*. [En línea]. Disponible en: https://arxiv.org

[7] D. Trautwein *et al.*, "Design and evaluation of IPFS: A storage layer for the decentralized web," in *Proc. ACM SIGCOMM 2022 Conf. (SIGCOMM '22)*, Amsterdam, Netherlands, Aug. 2022, pp. 739–752, doi: 10.1145/3544216.3544232.

[8] R. Patel, U. Biswas, S. Kodipaka, W. Carroll, P. Peranich, and M. Young, "A survey of recent advancements in secure peer-to-peer networks," *arXiv preprint arXiv:2509.19539*, Sep. 2025, doi: 10.48550/arXiv.2509.19539.

[9] F. O. Ehiagwina, N. A. Iromini, I. S. Olatinwo, K. Raheem, and K. Mustapha, "A state-of-the-art survey of peer-to-peer networks: Research directions, applications and challenges," *J. Eng. Nat. Sci. (JENRS)*, vol. 1, no. 1, pp. 19–38, Feb. 2022, doi: 10.55708/js0101003.

[10] Arsys, "Todo sobre la arquitectura cliente-servidor," *Arsys*. https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor

11. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar... Firmado: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar... Firmado: RAMIRO RENE RIVERA GUAMAN

Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC